

Model Context Protocol (MCP): Avaliação da Qualidade de Descrições de Tools com Base em Contexto de Código Fonte

Pedro H. P. Rodrigues¹, Vinicius R. A. de A. Moreira¹

¹Instituto de Ciências Exatas e Informática – Pontifícia Universidade Católica de Minas Gerais
Caixa Postal 500 – 30535-901 – Belo Horizonte – MG – Brazil

{vinicius,pedro}viniciusarantes133@gmail.com, pedrohenriquepr08@gmail.com

Resumo. O Model Context Protocol (MCP) permite que agentes de Inteligência Artificial (IA) interajam com sistemas externos por meio de tools, cujas descrições em linguagem natural orientam sua utilização. Entretanto, descrições de baixa qualidade, caracterizadas pela presença de smells, podem comprometer a eficácia desses agentes. Nesse contexto, este trabalho **realiza um estudo sobre a avaliação da qualidade dessas descrições por meio de uma rubrica multidimensional, da qual se derivam deficiências associadas a smells em ferramentas MCP**, incorporando o código das ferramentas ao processo de avaliação. Para isso, são minerados repositórios de servidores MCP no GitHub e realizadas avaliações automatizadas das descrições das tools. O objetivo é comparar resultados obtidos a partir das descrições com aqueles enriquecidos pelo código da ferramenta. Espera-se demonstrar que a inclusão desse contexto técnico revela inconsistências entre descrição e implementação, tornando mais precisa a avaliação de qualidade e **a identificação** desses smells.

Abstract. The Model Context Protocol (MCP) enables Artificial Intelligence (AI) agents to interact with external systems through tools, whose natural language descriptions guide their usage. However, poor-quality descriptions, characterized by the presence of smells, can compromise the effectiveness of these agents. In this context, this work **conducts a study on assessing the quality of these descriptions through a multidimensional rubric, from which deficiencies associated with smells in MCP tools are derived**, incorporating the tools' source code into the evaluation process. To achieve this, MCP server repositories are mined from GitHub, and automated evaluations of tool descriptions are performed. The objective is to compare results obtained solely from descriptions with those enriched by the tool's code. It is expected to demonstrate that incorporating this technical context reveals inconsistencies between description and implementation, thereby improving both the quality assessment and **the identification** of these smells.

Bacharelado em Engenharia de Software - PUC Minas
Trabalho de Conclusão de Curso (TCC)

Orientador de conteúdo (TCC I): João Pedro Batisteli - joaobatisteli@pucminas.br

Orientador acadêmico (TCC I): Leonardo Vilela - leonardocardoso@pucminas.br

Orientador do TCC II: Glender Brás - glendermedeiros@pucminas.br

Belo Horizonte, 28 de junho de 2026.

1. Introdução

O avanço dos modelos de linguagem (*Large Language Models*, LLMs) transformou o paradigma de interação entre humanos e máquinas [Hasan et al. 2026]. No entanto, a ausência inicial de padrões de comunicação entre LLMs e ferramentas externas gerou uma fragmentação técnica que motivou o surgimento do *Model Context Protocol* (MCP), este que é um protocolo aberto criado para padronizar a integração de modelos de IA com programas e fontes de dados externas. [Shi et al. 2026]. A adoção do protocolo tem crescido de forma acelerada, e um levantamento publicado no início de 2026 apontou que seus Kits de Desenvolvimento de Software (*Software Development Kit*, SDK) já registravam aproximadamente 97 milhões de *downloads* mensais [LLC 2026]. O principal diferencial do MCP reside no uso de ferramentas (*tools*) descritas em linguagem natural, que o agente pode selecionar e invocar em tempo de execução conforme a tarefa a ser resolvida. Esse funcionamento impõe, contudo, uma dependência crítica da qualidade documental, uma vez que o modelo não tem acesso ao código que implementa as ferramentas. As decisões de usá-las, como invocá-las e quais parâmetros fornecer são tomadas com base em suas descrições [Shi et al. 2026, Luo et al. 2025]. Dessa forma, assim como *code smells* comprometem a manutenibilidade de *softwares* tradicionais, descrições ambíguas ou incompletas comprometem o comportamento do agente, podendo induzir erros de planejamento, uso incorreto de parâmetros e alucinações [Mehedi Hasan et al. 2026].

Estudos recentes investigam a prevalência de anomalias (ou *smells*) em descrições de ferramentas MCP por meio de análise documental [Mehedi Hasan et al. 2026][Guo et al. 2025, Shi et al. 2026]. Essa abordagem avalia se a documentação comunica adequadamente o propósito, os parâmetros e as restrições de uso de uma ferramenta, examinando aspectos como clareza, completude e consistência, sem considerar a implementação técnica [Hasan et al. 2026]. Entretanto, evidências indicam que falhas relacionadas à configuração e à documentação representam 38,66% dos problemas observados em servidores MCP [Taraghi et al. 2026]. A elaboração e a manutenção dessas descrições dependem frequentemente de processos manuais desacoplados do código. Cabe destacar que, no MCP, a descrição não determina nem gera a implementação da ferramenta, mas atua como um artefato de documentação redigido manualmente para um código já existente, sendo este o que define o comportamento efetivo da ferramenta em execução. A prevalência de inconsistências entre descrição e implementação observada em servidores reais [Shi et al. 2026] evidencia que os dois artefatos podem divergir, o que motiva o uso do código como fonte adicional de evidência no processo de avaliação da qualidade da descrição. Por esse motivo, existe o risco de surgirem discrepâncias semânticas entre a documentação e a implementação real. Diante disso, existe uma lacuna na **análise da qualidade das descrições das**

ferramentas MCP, que, sem o contexto do código, pode não ser suficiente para a avaliação de sua qualidade.

A análise desse problema é relevante para a evolução sustentável de sistemas baseados em agentes de Inteligência Artificial (IA). Inconsistências semânticas podem reduzir a eficácia dos agentes e ampliar a superfície de exposição a vulnerabilidades, incluindo ataques de injeção de comandos [Taraghi et al. 2026]. Além disso, investigações recentes identificaram a presença de anomalias— **como ambiguidade de propósito, omissão de parâmetros e ausência de limitações declaradas (Seção 2)** — em grande parte das ferramentas disponíveis no ecossistema MCP [Hasan et al. 2026], e **relatórios de segurança alertam que essas mesmas ambiguidades documentais podem favorecer a execução de ações não autorizadas ou a exposição indevida de dados sensíveis** [National Security Agency 2026]. Nesse contexto, a incorporação de informações provenientes do código apresenta potencial para complementar abordagens tradicionais de avaliação de qualidade. Tal incorporação permite confrontar a descrição textual com a implementação efetiva da ferramenta, fornecendo evidências adicionais sobre possíveis inconsistências.

Tendo em vista essa lacuna, este trabalho tem como objetivo geral **investigar se a avaliação da qualidade de descrições de ferramentas MCP é influenciada pela disponibilização do código da ferramenta ao avaliador**. Para alcançar esse objetivo, definem-se os seguintes objetivos específicos: (i) **Examinar se o fornecimento do código da *tool* resulta em maior precisão na avaliação da qualidade de sua descrição**; e (ii) **Identificar os atributos de avaliação mais afetados pela adição do código da *tool***; e (iii) **Realizar uma análise estatística sobre os resultados obtidos nas avaliações**.

Esses objetivos são orientados pelas seguintes questões de pesquisa: **RQ1: o fornecimento do código da *tool* ao avaliador aumenta a precisão da avaliação da qualidade de sua descrição, em relação à avaliação baseada apenas na descrição?** **RQ2: quais atributos da rubrica de qualidade documental são mais afetados pela adição do código da *tool*?** **RQ3: as diferenças observadas entre os escores dos cenários com e sem código são estatisticamente significativas?**

Como parte da investigação, será conduzida uma análise sobre um conjunto de servidores MCP de código aberto, selecionados a partir de critérios de popularidade e maturidade do projeto. A avaliação irá se concentrar no confronto entre as descrições em linguagem natural das ferramentas e sua lógica de implementação, representada por informações estruturais extraídas do código. Espera-se que essa abordagem possibilite a identificação de inconsistências que não são facilmente observáveis por métodos exclusivamente textuais, como parâmetros implementados, mas não documentados, ou funcionalidades descritas sem correspondência evidente na implementação. Dessa forma, busca-se produzir evidências empíricas capazes de apoiar ou refutar a hipótese de que a incorporação do contexto técnico pode ampliar a capacidade de detecção de *smells* em ferramentas MCP. **Essa avaliação é operacionalizada por meio de uma rubrica de seis dimensões de qualidade documental, pontuadas em escala Likert de cinco pontos por modelos de linguagem atuando como avaliadores (*LLM-as-a-Judge*), dos quais se derivam as deficiências associadas aos *smells* investigados (Seção 4).**

As próximas seções deste artigo estão organizadas da seguinte forma. A Seção 2

apresenta a Fundamentação Teórica sobre MCP, qualidade no contexto de engenharia de software e detecção de *smells*. A Seção 3 discute os trabalhos relacionados. Por fim, a Seção 4 descreve os materiais e métodos empregados na condução do estudo.

2. Materiais e Métodos

Esta seção apresenta os procedimentos metodológicos adotados para a condução da pesquisa, bem como os instrumentos utilizados para coleta e análise dos dados. A pesquisa caracteriza-se como um estudo empírico, quantitativo e comparativo, cujo objetivo é investigar se a inclusão do código influencia a avaliação da qualidade de descrições de ferramentas MCP.

O fluxo metodológico adotado encontra-se resumido na Figura 1. O processo inicia-se com a coleta de repositórios MCP públicos, seguida pela extração das ferramentas e das informações sobre sua implementação. Em seguida, a qualidade das descrições das ferramentas é avaliada em dois cenários distintos: no primeiro, considerando a descrição em linguagem natural da ferramenta; no segundo, considerando a mesma descrição acompanhada pelo código correspondente. **Nos dois cenários avalia-se a mesma descrição em linguagem natural; a única variável entre eles é a presença ou ausência do código correspondente, e não a linguagem em si.** Por fim, os resultados obtidos são organizados e submetidos a uma análise comparativa e estatística.

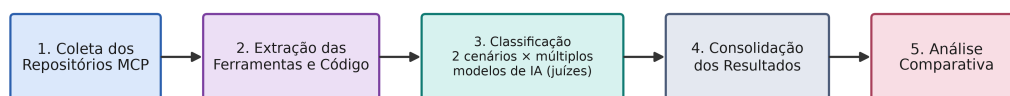


Figura 1. Fluxo metodológico adotado na pesquisa

2.1. Coleta dos Repositórios MCP

A primeira etapa consiste na construção do conjunto de dados que **serve** de base para os experimentos. Para isso, foram minerados repositórios públicos relacionados ao ecossistema MCP hospedados no GitHub. O objetivo dessa etapa é identificar implementações reais de servidores MCP que representem o cenário atual.

As consultas foram construídas a partir dos nomes dos pacotes necessários para a implementação do MCP. Para cada pacote, seu nome foi utilizado como termo de busca nos arquivos de dependências dos projetos. Essa etapa foi realizada por meio da busca de código da API REST do GitHub, que permite localizar repositórios com base em termos presentes no conteúdo de seus arquivos, ainda que a API limite cada consulta a um máximo de 1.000 resultados [Dong et al. 2023]. Dessa forma, foi necessário dividir a busca em consultas distintas, a fim de ampliar a quantidade de projetos identificáveis. Para isso, foi realizada uma consulta para cada combinação entre o nome da dependência e o respectivo arquivo de dependências em que ela poderia estar declarada. Como essa busca de código não devolve metadados do repositório em si, cada resultado foi complementado por uma consulta separada à API GraphQL do GitHub, de onde vieram o número de estrelas, a condição de *fork* e a linguagem principal usados nos filtros seguintes. Ao final, os repositórios duplicados foram unificados por identificador. Após isto, os que eram

fork e os que não atingiam o piso mínimo de estrelas, ou cuja linguagem principal não estava entre as linguagens-alvo do estudo, foram removidos.

Para garantir a relevância e a maturidade dos *softwares* analisados, a coleta foi restrita a repositórios que possuam no mínimo 100 estrelas (*stars*). A adoção desse limiar fundamenta-se em estudos de mineração de repositórios que identificam o número de estrelas como uma medida direta de popularidade, sendo 100 estrelas um marco estatístico comum para filtrar projetos experimentais ou de baixo valor agregado, conhecidos como *toy repositories* [Borges et al. 2016]. Ao final, foram selecionados os 206 servidores mais populares encontrados e que possuíam ao menos uma tool, dobrando a amostra utilizada no estudo de Hasan et al. (2026) para ampliar a cobertura do ecossistema investigado.

A Figura 2 detalha o funcionamento interno dessa etapa, implementada no script `run_step1.py`. A mineração é realizada via API REST de busca de código do GitHub, segmentada por termos: cada combinação entre um sinal de dependência (por exemplo, `@modelcontextprotocol/sdk`) e um arquivo de manifesto onde esse sinal poderia estar declarado (por exemplo, `package.json`) gera uma sub-consulta independente; a linguagem não faz parte dessa segmentação e só é determinada depois, na hidratação de cada candidato. Cada repositório encontrado é então hidratado via GraphQL para a obtenção de seus metadados de linguagem principal e número de estrelas, informações não disponíveis diretamente na busca de código. Em seguida, os repositórios são unificados por identificador (removendo duplicatas encontradas por mais de uma sub-consulta) e submetidos à verificação de elegibilidade descrita acima. Os repositórios aprovados são ordenados por número de estrelas, e os mais relevantes são extraídos para compor a amostra final.

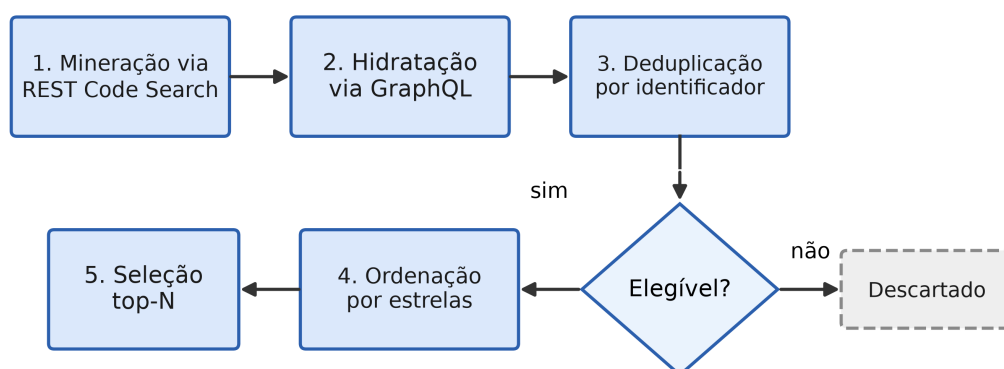


Figura 2. Funcionamento detalhado da Etapa 1 (Coleta dos Repositórios)

A busca por sinais de manifesto devolveu 12.058 candidatos brutos, todos provenientes de arquivos de dependência (`package.json`, `pyproject.toml`, `pom.xml` e equivalentes), únicos após a deduplicação por identificador. A Tabela 1 apresenta o funil de atrito aplicado sobre esse conjunto: o filtro de elegibilidade (repositório não ser *fork*, possuir pelo menos 100 estrelas e estar em uma das linguagens-alvo) reduziu esse conjunto para 723 repositórios.

Tabela 1. Funil de atrito da Etapa 1 (Coleta dos Repositórios MCP)

Etapa do filtro	Repositórios	% do bruto
Candidatos brutos (busca por manifesto)	12.058	100,0%
Aprovados no filtro (não-fork, estrelas $\geq$ mínimo, linguagem-alvo)	723	6,0%
Selecionados (qualificados pelo laço de <i>backfill</i>)	206	1,7%

Diferentemente de um corte simples pelas 206 posições mais bem colocadas, a seleção final passa por um laço de preenchimento (*backfill*): um repositório só permanece selecionado se, após a Etapa 2, confirmar pelo menos uma ferramenta; repositórios desqualificados são substituídos pelo próximo candidato do pool ainda não tentado, e o processo se repete até fechar as 206 vagas ou esgotar o pool. Entre os repositórios selecionados, a distribuição por linguagem principal está longe de ser uniforme (Figura 3).

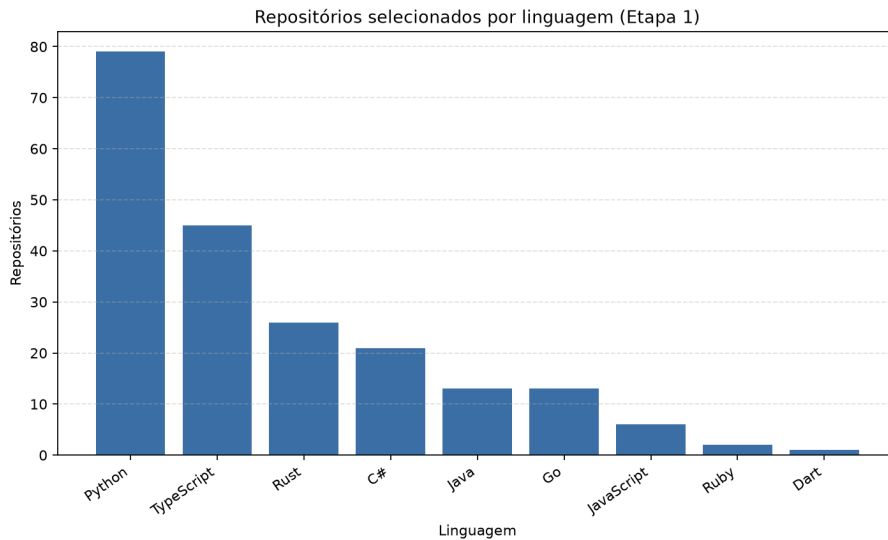


Figura 3. Repositórios selecionados por linguagem principal (Etapa 1)

Python (79, 38,3%) e TypeScript (45, 21,8%) concentram 60% da amostra, seguidos por Rust (26), C# (21), Java e Go (13 cada); JavaScript, Ruby e Dart somam, juntos, o restante. Nenhum repositório em Kotlin, PHP, Elixir, Lua ou Swift permaneceu na seleção final: os candidatos dessas linguagens que chegaram a ser selecionados foram descartados pelo laço de *backfill* por não confirmarem nenhuma ferramenta na Etapa 2 (Seção 4.2), ou nunca alcançaram o piso de estrelas dentro do pool.

2.2. Extração das Ferramentas e do Código Relacionado

Após a obtenção dos repositórios, cada um foi clonado localmente para permitir a identificação automática das ferramentas disponibilizadas por cada servidor MCP, chegando ao resultado de 12.171 ferramentas. Para isso, foi desenvolvido um algoritmo de mineração capaz de percorrer a estrutura dos projetos e localizar as implementações associadas às ferramentas expostas pelo servidor. Para cada ferramenta identificada, foram extraídos sua descrição em linguagem natural e sua localização no código. Um repositório

em que nenhuma ferramenta é localizada é tratado, na prática, como um falso positivo da etapa de coleta: ele é removido da amostra e substituído pelo próximo candidato ainda não testado, repetindo-se o processo até completar o número de repositórios definido para o estudo ou esgotar os candidatos disponíveis, *fechando o mesmo laço de backfill descrito na Seção 4.1*.

A Figura 4 detalha o funcionamento interno dessa etapa, implementada no script `run_step2.py` (clonagem, detecção de ferramentas e construção do grafo de chamadas); o laço de *backfill* entre Etapas 1 e 2 é implementado no script `run_backfill.py`. Cada repositório selecionado é primeiro clonado e então indexado estaticamente: são catalogadas todas as definições de função/método e, por arquivo, os *imports* declarados. Em seguida, aplica-se o padrão estrutural do SDK correspondente à linguagem do repositório para localizar as ferramentas declaradas; se nenhuma ferramenta é encontrada, o repositório é desqualificado e substituído, reentrando no fluxo a partir da clonagem.

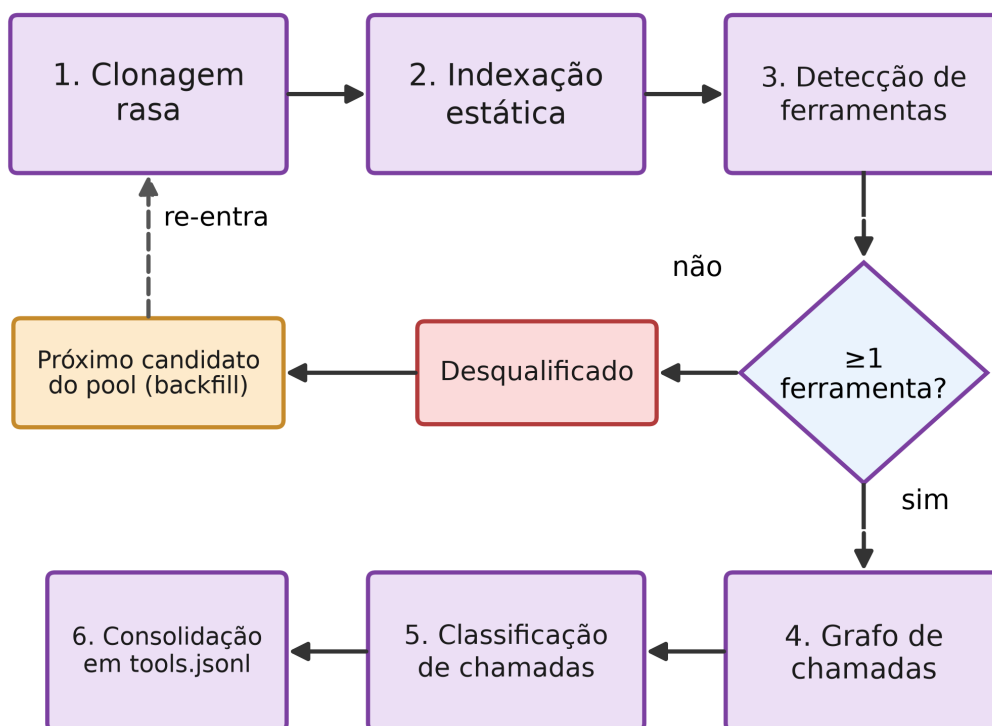


Figura 4. Funcionamento detalhado da Etapa 2 (Extração das Ferramentas)

Em seguida, foi realizada a extração do código relacionado a cada ferramenta. Como os servidores MCP possuem estruturas variadas, extrair apenas o método raiz de uma ferramenta pode não ser suficiente para capturar sua lógica, sendo necessário incluir também os métodos subsequentemente invocados. Para isso, foi construída uma árvore de chamadas (*call graph*) a partir do método inicial, *expandida recursivamente até um limite máximo de três níveis de profundidade*: o próprio método inicial constitui o primeiro nível; os métodos chamados diretamente por ele, *quando resolvidos*, formam o segundo nível; e as chamadas efetuadas a partir destes compõem o terceiro nível. *Esse*

limite de três níveis é um teto, não uma contagem garantida para toda ferramenta: uma parcela relevante das árvores para de se expandir antes do terceiro nível por não haver mais chamadas internas resolvíveis a partir daquele ponto. A limitação da profundidade a três níveis é uma decisão de engenharia, não uma medição prévia de onde termina a lógica relevante de uma ferramenta: ela contém o custo da análise (e do texto eventualmente enviado ao avaliador) e elimina, por construção, qualquer risco de recursão infinita, sem exigir detecção de ciclo separada (ver Apêndice A para o detalhamento completo da heurística de resolução). Um exemplo desse processo é apresentado na Figura 5.

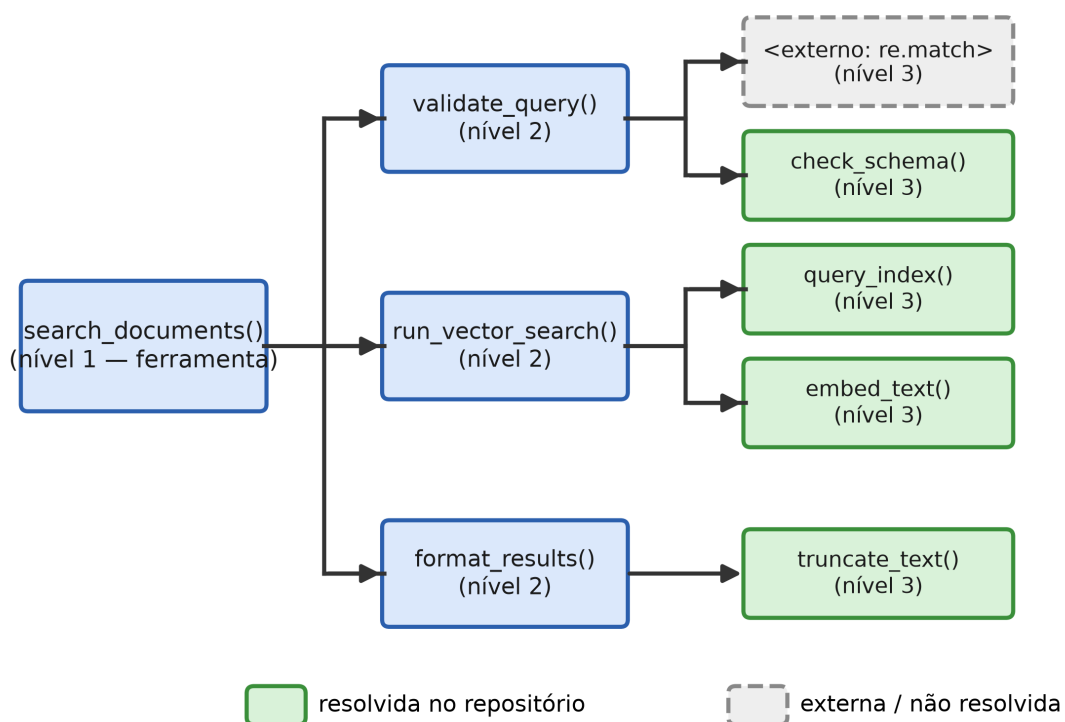


Figura 5. Exemplo de extração da árvore de chamadas de uma ferramenta

Ao final desta etapa, foi produzido um conjunto de dados estruturado contendo os servidores MCP analisados e as ferramentas identificadas em cada um deles. Para cada ferramenta, **são armazenadas** sua descrição em linguagem natural e o conjunto de métodos obtido a partir da árvore de chamadas. Esse conjunto de dados **serve** como entrada para as etapas subsequentes da pesquisa, nas quais as ferramentas dos repositórios coletados **são avaliadas** quanto à qualidade de suas descrições, com e sem o uso do código correspondente.

A Etapa 2 extraiu 12.171 ferramentas dos 206 repositórios selecionados; como a seleção final já incorpora o laço de *backfill*, todo repositório selecionado confirma, por construção, pelo menos uma ferramenta. A distribuição de ferramentas por linguagem (Figura 6) segue de perto a distribuição de repositórios vista na coleta.

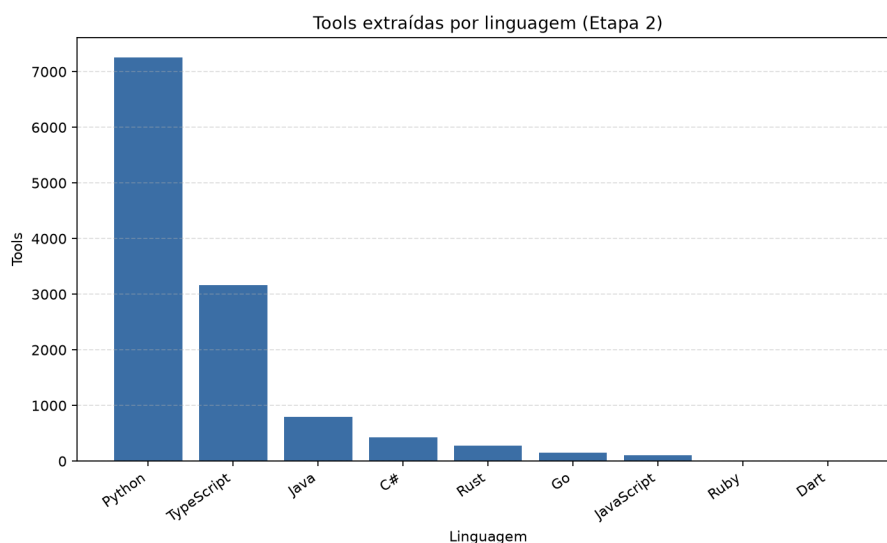


Figura 6. Ferramentas extraídas por linguagem (Etapa 2)

Python concentra 7.251 das 12.171 ferramentas (59,6%), seguido por TypeScript (3.162, 26,0%) e Java (793, 6,5%); as demais linguagens somam, juntas, pouco mais de 7% do total (Tabela 2).

Tabela 2. Repositórios selecionados e total de ferramentas extraídas, por linguagem

Linguagem	Repositórios	Tools (total)	Média/repositório
Python	79	7.251	91,78
TypeScript	45	3.162	70,27
Java	13	793	61,00
C#	21	423	20,14
Rust	26	276	10,62
Go	13	149	11,46
JavaScript	6	108	18,00
Ruby	2	5	2,50
Dart	1	4	4,00
Total	206	12.171	59,08

A concentração de ferramentas por repositório também é fortemente assimétrica: os três repositórios com mais ferramentas respondem, sozinhos, por 46,6% de todo o conjunto de dados (20,4%, 18,4% e 7,8%, respectivamente) — um deles em TypeScript, os outros dois em Python. Esse resultado eleva consideravelmente a média de ferramentas por repositório nessas duas linguagens (Tabela 2) e deve ser considerado ao interpretar médias agregadas por linguagem.

Quanto ao grafo de chamadas construído para cada ferramenta, 30,9% das chamadas (entre os níveis 2 e 3) foram resolvidas sem ambiguidade dentro do próprio repositório, 11,0% foram resolvidas por desempate entre candidatos ambíguos, e 58,1% permaneceram como chamadas externas ou dinamicamente determinadas — resultado

consistente com o fato de que boa parte da lógica invocada por uma ferramenta MCP reside em bibliotecas de terceiros ou na biblioteca padrão da linguagem, fora do escopo de resolução do algoritmo (ver Apêndice A).

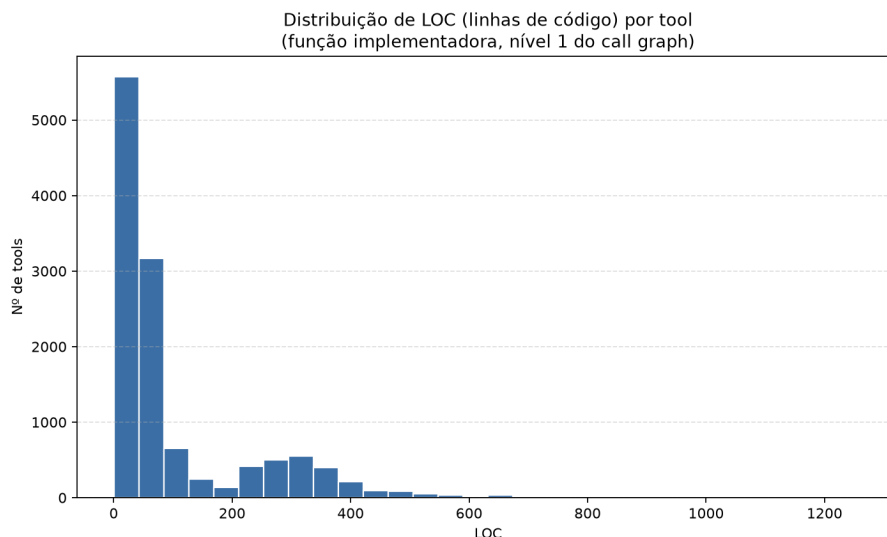


Figura 7. Distribuição de LOC (linhas de código) por ferramenta, função implementadora de nível 1

Por fim, 94,6% das descrições extraídas (11.512 de 12.171) correspondem a literais de *string* simples, sem necessidade de resolução adicional; a taxa varia por linguagem, de 100% em Dart e acima de 92% em Rust, TypeScript, Python, JavaScript e C#, até 85,9% em Java e 83,9% em Go. O Ruby é uma exceção notável, com apenas 40,0% (2 de 5) de descrições literais, ainda que sobre uma base amostral pequena.

Os gráficos e tabelas desta e da subseção anterior são gerados por um script de análise dedicado (`analysis-report.py`), a partir dos artefatos de saída já produzidos pelo pipeline, sem reprocessar os repositórios.

2.3. Classificação das tools por diferentes modelos de IA

Na terceira etapa, utilizando as *tools* coletadas nas etapas anteriores, é conduzido um experimento comparativo estruturado em dois cenários de avaliação, executados em diferentes modelos de IA. No primeiro cenário, as descrições das ferramentas são avaliadas com base em seu conteúdo em linguagem natural. No segundo, essas mesmas descrições são avaliadas em conjunto com informações extraídas do código correspondente. Essa estratégia permite identificar o impacto da inclusão do contexto de implementação na avaliação da qualidade das descrições.

A Figura 8 detalha o funcionamento interno dessa etapa, implementada no script `run_step3.py`, com execução paralela e rotação de chaves de API no script `run_parallel_step3.py`. Para cada ferramenta, são montados os dois cenários de avaliação descritos acima; cada cenário gera um *payload* contendo o nome da ferramenta, seu servidor de origem, a descrição e, quando aplicável, o código de contexto, enviado a cada juiz habilitado de forma concorrente. As respostas dos juízes podem resultar em uma avaliação concluída, em uma recusa de segurança do provedor, ou em um erro técnico

durante o processamento. As duas primeiras são gravadas no arquivo de resultados e recebem uma entrada de *checkpoint*, permitindo retomar a execução sem repetir avaliações já concluídas; o erro técnico não gera registro de resultado nem entrada de *checkpoint*, apenas um registro em um log de erros separado, e é reprocessado automaticamente na rodada seguinte.

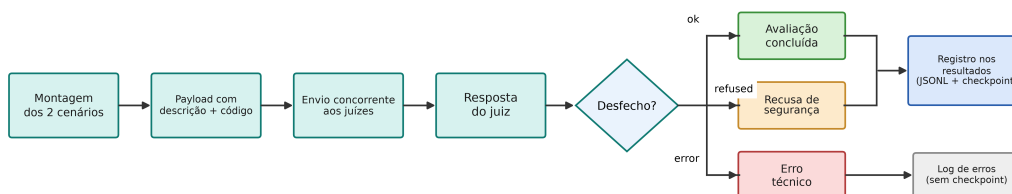


Figura 8. Funcionamento detalhado da Etapa 3 (Classificação das Ferramentas)

Em ambos os cenários, a avaliação é conduzida com base na rubrica proposta por Hasan et al. (2026), composta por seis componentes: *Purpose*, que avalia a clareza com que a descrição comunica a finalidade da ferramenta; *Guidelines*, que verifica se existem orientações sobre quando e como utilizá-la; *Limitations*, que analisa a explicitação de restrições, premissas ou casos de falha; *Parameter Explanation*, que mede a qualidade da documentação dos parâmetros de entrada; *Length and Completeness*, que avalia o nível de detalhamento e cobertura da descrição; e *Examples*, que verifica a presença de exemplos de utilização. Cada componente recebe uma pontuação em uma escala Likert de cinco pontos, sendo posteriormente utilizada para identificar deficiências associadas aos respectivos *smells* descritos pelos autores.

Em ambos os cenários, os modelos de linguagem atuam como o mesmo avaliador (*LLM-as-a-Judge*), recebendo as descrições das ferramentas e atribuindo as seis pontuações da rubrica acima. No primeiro cenário, essa avaliação é realizada sem qualquer acesso à implementação da funcionalidade analisada. No segundo cenário, o mesmo avaliador recebe adicionalmente o código correspondente—isto é, os trechos de código-fonte reunidos pela árvore de chamadas construída na Seção 4.2. Dessa forma, o avaliador pode confrontar a descrição fornecida com aspectos observáveis da implementação, verificando a existência de omissões, inconsistências ou informações potencialmente enganosas que não seriam perceptíveis apenas pela análise da descrição.

2.4. Consolidação dos Resultados por Cenário

Concluída a etapa de classificação, os resultados produzidos nos dois cenários experimentais são coletados e estruturados em uma base de dados unificada. Para cada ferramenta analisada, são armazenados identificadores que permitem sua associação ao servidor MCP de origem, bem como os resultados obtidos nos cenários com e sem a inclusão de informações provenientes do código, além de qual modelo de IA foi utilizado na avaliação.

A base de dados resultante contém os escores atribuídos pelo modelo para cada dimensão de qualidade avaliada, além das classificações finais produzidas em cada execução do experimento. Dessa forma, cada ferramenta possui dois conjuntos de resultados por modelo, diretamente vinculados à mesma descrição, diferindo apenas pela presença ou

ausência das informações extraídas do código. Essa organização **estabelece** um relacionamento direto entre as avaliações realizadas sobre uma mesma ferramenta, preservando o pareamento necessário para as análises estatísticas subsequentes e possibilitando a comparação consistente dos resultados obtidos nos dois cenários experimentais.

Concretamente, cada registro de avaliação (implementado no script `run_step3.py`) identifica a ferramenta por uma chave estável (`tooluid`, combinando repositório, nome qualificado e localização no código-fonte) e armazena o cenário (`description_only` ou `with_source`), o juiz responsável (identificador, provedor e modelo), o desfecho da chamada (concluída ou recusada) e, quando concluída, as seis pontuações da rubrica; recusas ficam sem pontuação, e erros técnicos, por não gerarem registro, ficam fora dessa base e são tratados à parte, em um log de erros. É essa combinação de `tooluid` e juiz, repetida nos dois cenários, que forma o par usado pelo Teste de Wilcoxon na análise seguinte. As tabelas e gráficos descritivos dessa base são gerados por um script de análise dedicado (`analysis_evaluation_report.py`).

2.5. Análise Comparativa dos Experimentos

Na etapa final, os resultados obtidos **são analisados** com o objetivo de identificar diferenças entre os dois cenários de avaliação. Essa análise busca validar se a utilização do código de fato altera os resultados da avaliação das descrições, além de identificar quais atributos da rubrica utilizada para avaliar a qualidade são mais e menos impactados pela adição do código.

Inicialmente, **realiza-se** uma análise por meio do cálculo de medidas como média, mediana e desvio-padrão dos escores atribuídos a cada dimensão de qualidade avaliada, para identificar se **há** diferença numérica nos resultados gerados com e sem o contexto do código das ferramentas MCP, atendendo ao objetivo específico (ii) desta pesquisa. A partir dessa comparação, é possível avaliar quais atributos **são** mais afetados pela inserção do contexto técnico. **Aplica-se** também o Teste de Postos Sinalizados de Wilcoxon [Wilcoxon 1945], **individualmente** para cada dimensão de qualidade, a fim de verificar se as diferenças observadas entre as avaliações são estatisticamente significativas, concluindo o objetivo específico (iii). O teste **é aplicado** separadamente para cada modelo de linguagem utilizado, permitindo verificar se o efeito da inclusão do código se mantém consistente entre diferentes avaliadores. Como as duas avaliações são realizadas sobre a mesma ferramenta, o teste é adequado por considerar amostras pareadas e não exigir a suposição de normalidade dos dados. Os resultados dessa análise **fornecem** as evidências empíricas necessárias para apoiar ou refutar a hipótese de que a incorporação do contexto técnico amplia a capacidade de detecção de *smells* em descrições de ferramentas MCP.

Formalmente, para cada dimensão da rubrica e cada juiz, testa-se a hipótese nula de que a distribuição das diferenças pareadas entre os escores dos cenários `with_source` e `description_only` é simétrica em torno de zero, contra a hipótese alternativa de que a inclusão do código desloca essa distribuição. Como o teste é repetido para as seis dimensões da rubrica em cada juiz, avalia-se a necessidade de correção para comparações múltiplas antes de reportar significância. Pares de ferramentas em que os dois cenários recebem exatamente o mesmo escore (diferença zero) são descartados do cálculo do teste, conforme o tratamento padrão de empates do Teste de Wilcoxon; a proporção de pares descartados por esse motivo é reportada junto com o resultado do

teste, por dimensão, como indicador adicional do quanto a inclusão do código muda ou não a nota atribuída. Essa análise é implementada no mesmo script de análise dedicado da subseção anterior, a partir da base consolidada ali descrita.

3. Resultados

Esta seção apresentará a análise comparativa entre os dois cenários avaliativos descritos na Seção 4: as estatísticas descritivas (média, mediana e desvio-padrão) por dimensão da rubrica, com e sem a inclusão do código, e o resultado do Teste de Postos Sinalizados de Wilcoxon aplicado a cada dimensão e a cada modelo de IA avaliador. Os resultados relativos à coleta dos repositórios e à extração das ferramentas, por serem parte da caracterização do conjunto de dados construído, encontram-se na Seção 4, junto à descrição do método que os produziu.

Referências

- Borges, H., Hora, A., and Valente, M. T. (2016). Predicting the popularity of github repositories. In *Proceedings of the 12th International Conference on Predictive Models and Data Analytics in Software Engineering*, pages 1–10.
- Dong, Y., Li, Z., Tian, Y., Sun, C., Godfrey, M. W., and Nagappan, M. (2023). Bash in the wild: Language usage, code smells, and bugs. *ACM Trans. Softw. Eng. Methodol.*, 32(1).
- Femmer, H., Méndez Fernández, D., Wagner, S., and Eder, S. (2017). Rapid quality assurance with requirements smells. *Journal of Systems and Software*, 123:190–213.
- Guo, H., Hao, Y., Zhang, Y., Xu, M., Lv, P., Chen, J., and Cheng, X. (2025). A Measurement Study of Model Context Protocol Ecosystem. *arXiv e-prints*.
- Hasan, M. M., Li, H., Fallahzadeh, E., Rajbahadur, G. K., Adams, B., and Hassan, A. E. (2026). Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers. *ACM Trans. Softw. Eng. Methodol.* Just Accepted.
- ISO/IEC (2011). Iso/iec 25010:2011 – systems and software engineering – systems and software quality requirements and evaluation (square) – system and software quality models. Technical report, International Organization for Standardization, Geneva, Switzerland.
- ISO/IEC/IEEE (2017). Iso/iec/ieee 24765:2017 – systems and software engineering – vocabulary. Technical report, International Organization for Standardization, Geneva, Switzerland.
- Khan, J. Y., Khondaker, M. T. I., Uddin, G., and Iqbal, A. (2021). Automatic detection of five api documentation smells: Practitioners’ perspectives. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 318–329.
- LLC, A. (2026). The model context protocol: A status report for enterprise customer experience leaders. White paper, Avaya.
- Luo, Z., Shen, Z., Yang, W., Zhao, Z., Jwalapuram, P., Saha, A., Sahoo, D., Savarese, S., Xiong, C., and Li, J. (2025). MCP-Universe: Benchmarking Large Language Models with Real-World Model Context Protocol Servers. *arXiv e-prints*.

- Mehedi Hasan, M., Li, H., Krishnan Rajbahadur, G., Adams, B., and Hassan, A. E. (2026). Model Context Protocol (MCP) Tool Descriptions Are Smelly! Towards Improving AI Agent Efficiency with Augmented MCP Tool Descriptions. *arXiv e-prints*.
- National Security Agency (2026). Model context protocol (mcp): Security design considerations for ai-driven automation. Cybersecurity report, NSA Cybersecurity Information.
- Pressman, R. S. and Maxim, B. R. (2016). *Engenharia de Software: Uma Abordagem Profissional*. AMGH, Porto Alegre, 8 edition.
- Shi, Y., Zhang, X., Zhang, X., Shen, X., Ouyang, H., Qiu, H., Zhang, M., and Yang, M. (2026). Description-code inconsistency in real-world mcp servers: Measurement, detection, and security implications. *arXiv preprint arXiv:2606.04769*.
- Taraghi, M., Morovati, M. M., and Khomh, F. (2026). Real faults in model context protocol (mcp) software: a comprehensive taxonomy. *arXiv preprint arXiv:2603.05637*.
- Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J., and Stoica, I. (2023). Judging llm-as-a-judge with mt-bench and chatbot arena. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S., editors, *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623. Curran Associates, Inc.

Apêndice

A. Algoritmo de Construção do Grafo de Chamadas e Composição do Contexto de Código

Este apêndice detalha o algoritmo desenvolvido para a construção do grafo de chamadas (*call graph*) de cada ferramenta MCP e para a composição do trecho de código usado como contexto adicional no cenário de avaliação que inclui código (Seção 4). O algoritmo é heurístico e baseado em análise estática via tree-sitter: a partir da função que implementa a própria ferramenta, resolve por nome as chamadas feitas dentro dela, até três níveis de profundidade, distinguindo chamadas resolvidas dentro do próprio repositório de chamadas externas ou dinamicamente determinadas, conforme ilustrado na Figura 5.

Procedimento de Extração

O algoritmo opera em três fases distintas, executadas uma vez por repositório:

1. **Indexação estática do repositório:** são construídos, a partir de todo o código do repositório, um índice de definições (nome simples e nome qualificado, `Classe.método`, de toda função/método encontrado, com referência ao seu corpo) e, por arquivo, um índice de *imports* (alias local usado no arquivo → módulo e nome original de onde veio). Ambos são heurísticas de escopo léxico, não uma resolução completa de símbolos.
2. **Construção recursiva do grafo de chamadas:** a árvore de cada ferramenta é construída a partir da função que a implementa (nível 1). Para cada chamada

encontrada no corpo de uma função, uma heurística de resolução por nome é aplicada em ordem, valendo a primeira que casar: (i) receptor `self/this` → método da própria classe; (ii) mesmo nome definido no mesmo arquivo → função auxiliar local; (iii) receptor casa com um alias de *import* conhecido → busca no módulo importado; (iv) nome único em todo o repositório → resolve diretamente; (v) não encontrado em lugar nenhum (biblioteca externa, *stdlib*, ou chamada genuinamente dinâmica) → tratado como folha externa, sem recursão. Uma chamada resolvida vira um novo nó, expandido recursivamente até o nível 3; nós de nível 3 continuam sendo chamadas normalmente resolvidas, mas suas próprias chamadas internas não são mais exploradas. Quando mais de um candidato válido é encontrado num mesmo passo (nome duplicado em escopos distintos), o desempate é feito pela menor distância de diretório até o arquivo de origem da chamada, e o nó resultante permanece explicitamente sinalizado como *ambíguo* na estrutura de dados, em vez de ser tratado como uma resolução certa.

3. **Composição do contexto de código:** no cenário de avaliação que inclui *código*, a árvore é percorrida em pré-ordem e, para cada nó resolvido, o trecho de *código* correspondente (linhas de início e fim exatas) é lido diretamente do repositório clonado em disco e anotado com um comentário indicando o nível na árvore, a chamada de origem e a localização exata (arquivo:linhas). Nós externos/não resolvidos entram apenas como um comentário sinalizando a chamada não resolvida, sem trecho de código. Os trechos são concatenados na ordem de visita; o texto resultante é limitado a um número máximo de caracteres antes de ser enviado ao avaliador, para conter o custo e a atenção do modelo: um corte por tamanho, não uma sumarização do conteúdo.

Pseudocódigo do Algoritmo

A lógica de construção do grafo e de resolução de chamadas é apresentada na Listagem 1. A profundidade máxima é imposta pela própria recursão: ao atingir o nível limite, a função retorna sem procurar novas chamadas, o que também elimina, por construção, qualquer risco de loop em recursão direta ou mútua, sem exigir detecção de ciclo separada.

```
1 MAX_LEVEL = 3
2
3 def build_call_graph(fn_def, level=1):
4     node = Node(level, resolved=True, external=False,
5                 ambiguous=False, location=fn_def.location)
6
7     if level >= MAX_LEVEL:
8         return node # n de nivel 3: resolvido, mas nao
9                     expandido
10
11     for call_site in extract_calls(fn_def.body):
12         resolved_def, ambiguous = resolve_call(call_site, fn_def
13         )
14
15         if resolved_def is None:
16             node.calls.append(
17                 ExternalLeaf(level + 1, call_site.raw_text))
```

```

16         continue
17
18         child = build_call_graph(resolved_def, level + 1)
19         child.ambiguous = ambiguous
20         child.raw_call_text = call_site.raw_text
21         node.calls.append(child)
22
23     return node
24
25
26 def resolve_call(call_site, fn_def):
27     name = call_site.callee_name
28
29     # 1. receptor self/this -> metodo da propria classe
30     if call_site.receiver in ("self", "this"):
31         candidate = definitions.by_qualified_name.get(
32             f"{fn_def.class_name}.{name}")
33         if candidate:
34             return candidate, False # (definicao, ambiguous)
35
36     # 2. mesmo nome, mesmo arquivo
37     # 3. receptor casa com alias de import conhecido
38     # 4. nome unico em todo o repositorio
39     candidates = lookup_by_scope(name, call_site, fn_def)
40     if len(candidates) == 1:
41         return candidates[0], False
42     if len(candidates) > 1:
43         return nearest_by_directory(candidates, fn_def.file),
44             True
45
46     # 5. nao encontrado -> chamada externa/dinamica
47     return None, False

```

Listing 1. Construção do grafo de chamadas e resolução por nome

Justificativa Técnica

A limitação a três níveis é uma decisão de engenharia, não uma medição prévia de onde a lógica relevante de uma ferramenta termina: ela contém o tamanho da árvore (e, por consequência, do texto eventualmente enviado ao avaliador) a um custo previsível, e elimina, por construção, qualquer risco de loop infinito em recursão direta ou mútua, sem exigir detecção de ciclo separada. A resolução de chamadas é heurística e baseada em nome, não uma checagem de tipos ou resolução semântica completa: um *trade-off* assumido dado o escopo do projeto. Por isso, toda chamada resolvida por desempate entre múltiplos candidatos válidos permanece marcada como ambígua na estrutura de dados final, em vez de ser silenciosamente tratada como uma resolução certa, preservando essa incerteza para análises posteriores.